AI APEX PARAGON

Codex Job-Focused Blueprint

From Python foundations to production AI systems, RAG, agents, evaluation, deployment, and a high-paying AI engineering job.

Student	Adarsh - AI Apex Paragon
Teacher	Codex
Duration	12 to 14 weeks full-time
Goal	Job-ready AI Engineer / GenAI Engineer by July-August 2026
Core Strategy	3 excellent portfolio projects, daily commits, DeepML + DSA in parallel
North Star	ENTITY - a local, private, voice-controlled AI PC agent

Rule: We do not learn everything. We learn what compounds into job skill, portfolio proof, and interview confidence.

Daily Operating System

Time	Track	Outcome
6:00 - 8:00 AM	DSA or DeepML	Interview patterns or ML math muscle
8:00 AM - 1:00 PM	Main AI Engineering	Best five hours for coding and building
2:00 - 4:00 PM	Concepts + implementation	Understand, code, debug, explain
4:00 - 6:00 PM	Project work	Portfolio features, docs, tests, deploys
7:00 - 8:00 PM	Review + GitHub	Notes, explain-back, commit, push

Parallel Tracks

- **Project 3:** Main AI Engineering - Python, APIs, SQL, FastAPI, LLMs, RAG, agents, deployment.
- **Project 4:** DeepML - vector math, NumPy, metrics, regression, similarity, embeddings.
- **Project 5:** DSA - arrays, strings, hash maps, two pointers, recursion, trees, graphs, interview explanations.

Execution Loop

Understand - Code - Break - Debug - Explain - Commit - Push - Publish.

Phase 0 - Current Checkpoint And Week 2 Patch

You have already built beginner Python, Git, files, JSON, APIs, Pandas practice, merging, pivot tables, Matplotlib, Seaborn, and charts. We do not restart. We patch gaps and move forward.

Week 2 Completion Sprint

- Add NumPy linear algebra: arrays, vector operations, dot product, matrix multiplication, cosine similarity.
- Add statistics basics: mean, median, standard deviation, correlation, probability counts.
- Build a real EDA project folder with dataset, cleaned analysis, charts, findings, and README.
- Clean minor script issues and push the repo.

Exit Criteria

- Can explain DataFrame cleaning, grouping, merging, pivoting, and charting.
- Can implement dot product and cosine similarity in NumPy.
- Can present 5 clear findings from a real dataset.

Phase 1 - Foundation And Data

Week	Focus	Topics	Output
1	Python + Git + APIs	Loops, dictionaries, functions, files, JSON, requests, venv, GitHub	Clean foundation repo with API scripts
2	NumPy + Pandas + EDA	Arrays, linear algebra, statistics, cleaning, merge, groupby, pivots, charts	Published EDA project

This phase makes you fluent enough to handle data and API-shaped objects without fear. It is the base layer for every AI system.

Phase 2 - Classical Machine Learning

Week	Focus	Topics	Output
3	ML Fundamentals	Train-test split, regression, classification, trees, random forest, metrics	Supervised ML project
4	ML Project + Kaggle	Feature engineering, encoding, scaling, cross-validation, tuning, model comparison	Kaggle-style project with README

- DeepML alignment: metrics, MSE, accuracy, precision, recall, vector operations.
- Interview alignment: explain bias, variance, overfitting, leakage, and why a metric was chosen.
- Portfolio rule: one polished notebook beats five unfinished experiments.

Phase 3 - Backend For AI

Week	Focus	Topics	Output
5	SQL + FastAPI	SQL basics, SQLite/Postgres, REST APIs, Pydantic, CRUD, database-backed APIs	Database-backed API project
6	Production Python	Env vars, logging, pytest, Docker basics, background tasks, deployment	Dockerized FastAPI mini-service

This is where you start looking like an engineer, not just an ML learner. Most GenAI products are backend systems with LLM calls inside.

Phase 4 - LLM Engineering And RAG

Week	Focus	Topics	Output
7	LLM APIs	Prompting, structured outputs, streaming, tool calling, retries, cost tracking	LLM assistant API
8	RAG Fundamentals	Embeddings, chunking, vector search, metadata, citations, document ingestion	PersonaDoc RAG begins
9	RAG Production	Hybrid search, reranking, hallucination control, eval datasets, FastAPI integration	PersonaDoc production

- RAG must include citations, source tracking, chunk strategy, and failure cases.
- Evaluation is mandatory: golden questions, retrieval checks, answer checks, latency and cost.
- PersonaDoc is the first public-grade portfolio project.

Phase 5 - Agents, Evaluation, And LLMOps

Week	Focus	Topics	Output
10	AI Agents	Tools, function calling, planning loops, memory/state, human approval, failure recovery	Agent prototype
11	LLMOps + Evals	Prompt versioning, golden datasets, LLM-as-judge, traces, latency, cost, safety	Portfolio Project 2
12	Production Agent	Tools, memory, APIs, deployment, UI/logs, case study	Portfolio Project 3

A real agent is not a chatbot. It is an LLM plus tools, memory, state, guardrails, logs, evaluation, and recovery behavior.

Phase 6 - Portfolio And Job Attack

Week	Focus	Actions	Output
13	Portfolio Polish	READMEs, architecture diagrams, demo videos, deployment, case studies	Three job-ready projects
14	Job Attack	Resume, LinkedIn, outreach, mock interviews, AI system design, daily applications	Applications and interviews

- Apply before you feel ready. Readiness grows through interviews.
- Every project needs a business problem, architecture, screenshots, demo, tradeoffs, and what you would improve.
- Target roles: AI Engineer, GenAI Engineer, LLM Application Engineer, ML Engineer, AI Backend Engineer.

The 3 Main Portfolio Projects

Project	What It Proves	Required Features
1. PersonaDoc RAG	You can build document intelligence systems	Upload/ingest docs, chunking, embeddings, vector search, citations, evals,
2. Multi-LLM Router + Eval Dashboard	You understand cost, latency, quality, fallbacks, and model selection	Model routing rules, multiple providers, token/cost logging, eval set, dashboard, fallback logi
3. Agentic Workflow App	You can build agents that do useful work, not just chat	Tool use, memory/state, human approval, logs, error handling, deployment

Quality rule: 3 excellent projects beat 30 small scripts. Each project must be understandable by a recruiter and respected by an engineer.

ENTITY - The North Star

ENTITY is the advanced capstone: a local, private, voice-controlled PC agent. It is not the first job project. It is the final boss that our portfolio components prepare us to build.

Component	Technology Direction	When We Build It
Voice	Whisper local speech-to-text	After RAG and agents
Brain	Local LLM via Ollama or similar	After LLM API mastery
Memory	RAG over notes, history, documents	During PersonaDoc
Tools	Python functions and APIs	During agent phase
Eyes	Screenshots and computer vision	Advanced extension
Hands	Safe OS automation	Advanced extension with guardrails

ENTITY build order: RAG memory - tool calling - agent state - local LLM - voice - vision - OS control.

The Code Of Paragon

- **Rule 1 - Ship weekly.** Every week ends with something committed, documented, or deployed.
- **Rule 2 - Explain it back.** If you cannot explain it simply, you do not own it yet.
- **Rule 3 - Daily commit ritual.** Code, commit, push. Momentum becomes visible proof.
- **Rule 4 - DeepML stays small but daily.** 20 to 60 minutes. It sharpens the math behind AI.
- **Rule 5 - DSA is targeted.** Enough to pass interviews, not enough to derail AI engineering.
- **Rule 6 - Evaluation is not optional.** Every serious LLM project needs tests, evals, logs, and failure analysis.
- **Rule 7 - Rest protects ambition.** Sleep is part of memory, judgment, and speed.
- **Rule 8 - ENTITY is the north star.** We build toward it piece by piece without letting it consume the job path.

The blueprint is not the victory. The commits, projects, interviews, and shipped systems are the victory.